
Entropy & Compression Based Evaluation on LLM Tokenizers

Darby Brill², Arnav Mazumder¹, Vattanary Tevy³, Erik Uri^{1*}

¹Paul G. Allen School of Computer Science and Engineering

²University of Washington, Department of Mathematics

³University of Washington, Department of Mechanical Engineering
{dbrill, am2107, vtevy, eksu}@uw.edu

Project Summary

Project Scope

This project reproduces and extends Erdogan et al.’s(4) information-theoretic analysis of LLM tokenizers. The original paper studies tokenizers as structured compressors and argues that tokenizer training scale redistributes entropy: unigram entropy increases while higher-order conditional entropies decrease or stabilize. It also argues that tokenizer improvements are fragile under train-test domain mismatch. We reproduce these claims for BPE(11), Unigram(6), WordPiece(10), and WordLevel(1) tokenizers, then extend the evaluation to SuperBPE(8), a tokenizer that can merge across whitespace boundaries. Our main goal is to test whether SuperBPE changes the compression-entropy tradeoff and whether its cross-whitespace tokens make it more sensitive to domain mismatch.

Methodology

We trained tokenizers across multiple vocabulary sizes, training scales, tokenizer families, and domains. The main tokenizer families were BPE, Unigram, WordPiece, WordLevel, and SuperBPE. We evaluated vocabulary sizes of 16k and 64k and tokenizer training sizes from 10^3 to 10^8 characters where data availability allowed. We used C4(9) for English web text, CodeParrot(3) for code, and GSM8K(2) for math. For each tokenizer, we computed compression ratio, empirical k -gram entropy for $k \in 1, 2, 3, 4, 5$, and entropy rate in bits per character. We also evaluated train-test domain mismatch by training tokenizers on English or code and testing them on other domains.

Results

Our results broadly support the original paper’s findings. In matched-domain settings, increasing tokenizer training scale generally improves compression and redistributes entropy: H_1 increases while higher-order conditional entropies decrease or stabilize. SuperBPE achieves strong compression at larger training scales, especially with a 64k vocabulary, but it also produces substantially higher unigram entropy. Its higher-order entropies remain comparable to BPE and WordPiece rather than uniformly lower. Under domain mismatch, SuperBPE was not as fragile as we initially expected; it often tracked BPE and WordPiece closely on code and Turkish, although its behavior on Chinese was less stable. Overall, SuperBPE appears to induce a different entropy-compression tradeoff rather than a strict improvement over standard tokenizers.

What was Easy

The easiest part was applying the information-theoretic evaluation once tokenized outputs were available. Compression ratio, empirical k -gram entropy, and entropy rate are tokenizer-agnostic metrics, so the same analysis could be applied across tokenizer families and domains. It was also relatively straightforward to compare SuperBPE with the other learned tokenizers because the evaluation framework did not need to change.

What was Difficult

The hardest part was producing the results in a timely manner. Evaluating each set of tokenizers on test datasets of 10M characters frequently yielded computation time of over an hour. This led to us having to plan our experiments out and

* Author names are listed alphabetically by last name. Code is available at <https://github.com/darbybrill/CSE-599-Project>.

allocate time to make sure we could achieve the results we wanted for our project. Additionally, evaluation on GSM8K as a test set was impossible due to it only containing approximately 3M characters despite needing 10M characters for evaluation. This size constraint also caused only partial results to be generated for tokenizers that were trained on the GSM8K dataset.

1 Introduction

Tokenization is a foundational preprocessing step in every modern language model pipeline. However, its downstream effects on model behavior remain underexplored. A tokenizer determines how raw text is segmented into discrete units before processed by a model, such as decisions that shape vocabulary size, sequence length, and what patterns a model can efficiently learn. The dominant architectures (Byte Pair Encoding, Unigram, WordPiece, and WordLevel) were developed under practical engineering constraints (11; 6; 10; 1), and their information-theoretic properties across diverse text domains have recently undergone rigorous analysis.

The paper we reproduce, Erdogan et al. (4), frames tokenizer quality using conditional entropy. Rather than evaluating tokenizers solely by downstream task performance, the authors measure how well a tokenizer compresses text. Specifically, how predictable each token is given its context. Their key findings are that increasing tokenizer training scale redistributes entropy: unigram entropy tends to increase, while higher-order conditional entropies decrease, and these gains degrade under train-test domain mismatch. This provides a model-agnostic benchmark for comparing tokenizers.

Our project replicates and extends the framework to SuperBPE (8), a tokenizer that allows merges across whitespace to form multi-word tokens. We evaluate all five tokenizer families on three domains, English web text (C4) (9), math problems (GSM8K) (2), and Python code (CodeParrot)(3), across vocabulary sizes of 16k and 64k and training scales spanning six orders of magnitude.

We find that SuperBPE is competitive with or exceeds standard BPE in compression at large training scales, exhibits higher unigram entropy, and is more robust to domain mismatch than our initial hypothesis predicted. Together, these results confirm the entropy-redistribution finding of the paper, extend it to a new tokenizer family, and show the tradeoff between per-token unpredictability and per-character information density.

2 Scope of the Project

Our project is a replication and extension of Erdogan et al.’s information-theoretic analysis of LLM tokenizers (4). The scope is limited to testing how tokenizer training scale, train-test domain mismatch, and tokenizer choice affect compression and empirical token n -gram entropy. The original paper frames tokenizers as structured compressors and studies how tokenization affects both sequence length and the statistical structure of the resulting token stream (4). We first reproduce two central qualitative findings from the original paper using learned tokenizer families, specifically BPE, Unigram, WordPiece, and WordLevel. We then extend the same evaluation framework to the SuperBPE tokenizer, one that merges across whitespace, with additional evaluation on structured domains such as code and math.

2.1 Addressed Claims from the Original Paper

1. **Claim 1: Increasing tokenizer training scale redistributes entropy.**

Erdogan et al. (4) find that as tokenizer training data increases, unigram entropy tends to increase while higher-order empirical conditional entropies decrease. In other words, tokenizers trained on larger datasets induce token streams with a more diverse marginal token distribution, but tokens become more predictable given short local context. We test this claim by training BPE, Unigram, WordPiece, and WordLevel tokenizers at increasing training data sizes and measuring unigram entropy and higher-order empirical conditional entropies on held-out matched-domain data.

This claim is supported if, as tokenizer training scale increases, unigram entropy rises while higher-order conditional entropies decrease on matched-domain test data.

2. **Claim 2: Tokenizer improvements are fragile under train-test domain mismatch.**

Erdogan et al. (4) find that tokenizer gains are distribution-dependent: when tokenizers are trained on English and evaluated on mismatched domains such as Turkish, Chinese, or code, compression does not consistently improve with larger tokenizer training sets, and higher-order empirical conditional entropies remain non-negligible. We reproduce this mismatch setting by training tokenizers on one source domain and evaluating them on different target domains. As an extension, we also include math as an additional structured target domain.

This claim is supported if increasing tokenizer training size improves compression or lowers conditional entropy more reliably in matched-domain evaluation than in mismatched-domain evaluation, and if mismatched-domain conditional entropies remain non-negligible.

2.2 New Hypotheses Tested in Our Extension

1. Hypothesis 1: SuperBPE may be more sensitive to domain mismatch than standard BPE.

SuperBPE extends standard BPE by allowing tokens to span across whitespace or word boundaries, producing larger “superword” units. This can improve compression on matched-domain text by capturing recurring multi-word patterns or domain-specific structures. However, because these larger units may encode patterns that are less transferable across domains, we hypothesize that SuperBPE may lose more of its advantage under train-test domain mismatch than standard BPE.

This hypothesis is supported if SuperBPE improves compression or conditional entropy on matched-domain test data, but its performance degrades more sharply than standard BPE when evaluated on mismatched domains.

2. Hypothesis 2: SuperBPE exhibits higher unigram entropy and lower higher-order entropies compared to other tokenizers.

Since SuperBPE can form tokens that span multiple words or whitespace-separated units, it may produce a larger set of more specific phrase-level tokens than standard tokenizers. This can spread probability mass across many specialized units, making individual tokens harder to predict and increasing unigram entropy H_1 . At the same time, because these larger tokens already capture local phrase structure, the remaining token sequence may become more predictable, leading to lower higher-order conditional entropies.

This hypothesis is supported if SuperBPE has higher H_1 than standard BPE, Unigram, and WordPiece, while achieving lower or comparable H_2, H_3, H_4 , and H_5 . This would suggest that SuperBPE shifts more structure into individual tokens while preserving predictable sequential structure across tokens.

Together, these claims define a focused empirical scope. The replication component checks whether our implementation recovers the original paper’s main qualitative findings about entropy redistribution and domain mismatch. The extension component asks whether newer or commonly used tokenizers change these conclusions, especially on structured domains such as code and math.

3 Methodology

3.1 Datasets and Preprocessing

We evaluate the tokenizers on three datasets covering english, math, and code. Specifically we use Colossal Clean Crawled Corpus (C4) for web text, GSM8K for math, and CodeParrot for code. These data sets are all sourced from Hugging Face (7). All datasets are preprocessed prior to tokenization. First, all data is normalized using standard NFKC Unicode norm (12). It then is put through a basic whitespace-based pre-tokenization. Finally, a set of special tokens: $\{\langle \text{UNK} \rangle, \langle \text{PAD} \rangle, \langle s \rangle, \langle /s \rangle\}$ where $\langle \text{UNK} \rangle$ is the unknown token, $\langle \text{PAD} \rangle$ is the padding token, $\langle s \rangle$ is the start of sentence token, and $\langle /s \rangle$ is the end of sentence token is introduced to the data sets.

We then partition each dataset into train, validation, and test sets. Each tokenizer vocabulary is trained exclusively on the training set and evaluated on the validation and test sets.

3.2 Tokenizer Preparation

For each of our tokenizers, we evaluate their entropy across varying configurations of vocabulary size, training scale, and domain. Specifically, we train learned tokenizer families including BPE, Unigram, WordPiece, WordLevel, and SuperBPE across different vocabulary sizes, training data sizes, and source domains. All of these tokenizers besides SuperBPE are sourced from Hugging Face (5). For SuperBPE we used the official GitHub to source our tokenizers.

Each tokenizers set is trained using different vocabulary sizes $|\mathcal{V}|$, and different training data sizes $|\mathcal{T}|$ measured in number of tokens. We evaluate vocabulary sizes of

$$|\mathcal{V}| \in \{16000, 64000\}$$

and training data sizes of

$$|\mathcal{T}| \in \{10^3, 10^4, 10^5, 10^6, 10^7, 10^8\}.$$

These choices of $|\mathcal{V}|$ and $|\mathcal{T}|$ allow for analysis of how the entropy of each tokenizer changes as a function of training data size and vocabulary size. For SuperBPE tokenizers we utilized a 78.125%, 0.21875% split between BPE tokenization and white space merges. We selected these values based on existing pretrained tokenizers and our desire to have sufficient white space merges to evaluate the effectiveness of SuperBPE.

3.3 Data Collection and Analysis

For each tokenizer group evaluated we collected the empirical token frequency distributions from tokenized test sequences of either 1M or 10M characters. Given a tokenized test sequence, $T = (t_1, t_2, \dots, t_n)$, we evaluated the empirical frequency of token t as $\hat{p}(t)$. To capture more structure we made a sliding window of length k for each order $k \in \{1, 2, 3, 4, 5\}$ to construct counts of all k length token sequences $(t_i, \dots, t_{i+k-1})$. From these counts we can calculate the conditional distributions $\hat{p}(t_i | t_{i-k+1}^{i-1})$. Following the empirical k -gram entropy estimator used by Erdogan et al. (4), we then computed

$$\hat{H}_k = -\frac{1}{n} \sum_{i=1}^n \hat{p}(t_i | t_{i-k+1}^{i-1}) \log \hat{p}(t_i | t_{i-k+1}^{i-1})$$

for $k = 1$, this reduces to empirical unigram entropy.

$$H_1 = -\sum_{i=1}^n \hat{p}(t_i) \log \hat{p}(t_i).$$

All logarithms are base 2, so entropy is measured in bits. In addition to empirical entropy, we measure compression using tokenized sequence length and bits-per-character on held-out text. For a tokenizer T and test distribution $\mathcal{D}$ we calculate the compression ratio of T over $\mathcal{D}$ by evaluating

$$CR(T; \mathcal{D}) = \frac{\sum_{x \in \mathcal{D}} |x|_{\text{UTF-8}}}{\sum_{x \in \mathcal{D}} |T(x)|}.$$

This lets us compare whether a tokenizer reduces the number of tokens needed to represent a dataset and whether this compression gain corresponds to lower higher-order conditional entropy.

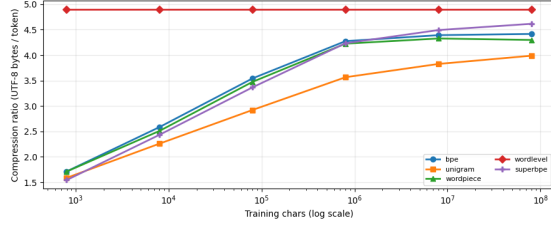
We then assess the tokenizers performance on a domain mismatch between the training and test datasets. To accomplish this we first train tokenizers on data from one data set and evaluate the compression ratio and empirical entropy of the tokenizer being tested on datasets in a different domain than what the tokenizer was originally trained on. For tokenizers trained on English we evaluated them on Chinese, Turkish, and code, and for tokenizers trained on code we evaluated them on English, Chinese, and Turkish. We were not able to test or train tokenizers on math for these experiments as there was not enough data in the GSM8K dataset to conduct conclusive experiments.

4 Results

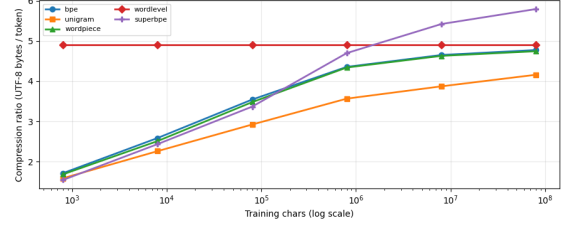
4.1 Compression

We first evaluate compression on matched-domain held-out data. Following Erdogan et al. (4), we measure compression ratio in UTF-8 bytes per token, so higher values indicate that each token represents more text. Figure 1 shows the results for English (C4), GSM8K math, and CodeParrot code at vocabulary sizes 16k and 64k.

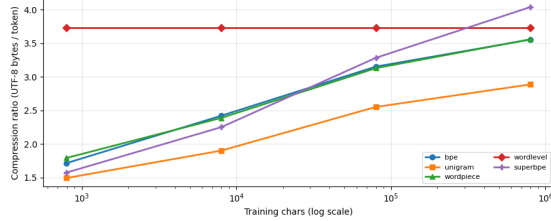
Across domains, we recover the expected scaling trend for the learned subword tokenizers: BPE, Unigram, WordPiece, and SuperBPE generally improve as tokenizer training size increases. In contrast, WordLevel remains nearly flat across scales, since it already stores whole words and therefore does not benefit much from additional training data in the same way. Among the learned tokenizers, Unigram is usually the weakest, while BPE and WordPiece are consistently strong baselines.



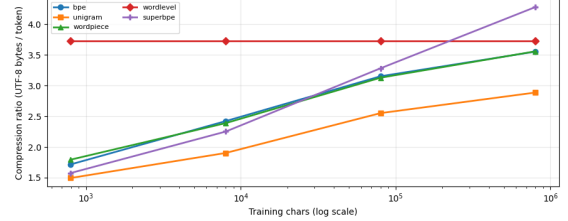
(a) English, 16k-vocab size



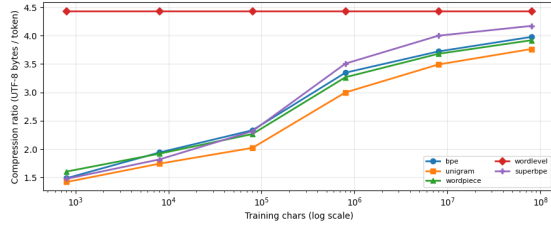
(b) English, 64k-vocab size



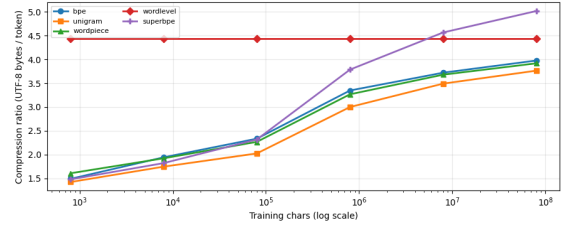
(c) GSM8K, 16k-vocab size



(d) GSM8K, 64k-vocab size



(e) CodeParrot, 16k-vocab size



(f) CodeParrot, 64k-vocab size

Figure 1: Compression ratio on matched-domain held-out data for English, math, and code. Higher values indicate that each token represents more bytes of text.

Our extension adds SuperBPE to this comparison. SuperBPE is competitive with BPE and WordPiece at small scales and often becomes the strongest learned tokenizer at larger scales, especially in the 64k setting. This is most visible on English and math, where SuperBPE slightly exceeds the other learned tokenizers at the largest training sizes, and on code, where it catches up and then surpasses BPE and WordPiece as scale increases. These results support the view that allowing merges across whitespace can improve compression by capturing recurring multi-word or structured patterns.

Overall, the compression results support the original claim that increasing tokenizer training scale improves compression for learned tokenizers, while also showing that SuperBPE changes the tradeoff in a useful way. It achieves strong compression, often matching or exceeding standard BPE and WordPiece at larger scales, though not uniformly at every scale or domain. The 64k setting generally strengthens these trends, suggesting that SuperBPE benefits especially from having both a larger vocabulary and more tokenizer training data.

4.2 k-gram Entropy

We next evaluate how tokenizer training scale affects the empirical k -gram entropy of the induced token sequences. Following Erdogan et al. (4), we report both entropy per token, $\hat{H}_k$, and entropy rate in bits per character. Entropy per token measures how unpredictable the next token is, while entropy rate normalizes by how much text each token represents. This distinction matters because a tokenizer can produce high-entropy tokens while still reducing uncertainty per character.

Figure 2 shows the 16k-vocabulary k -gram entropy results for English (C4), GSM8K math, and CodeParrot code. Across BPE, Unigram, WordPiece, and WordLevel, we recover the main entropy redistribution pattern from Erdogan et al. (4): as tokenizer training scale increases, unigram entropy H_1 generally rises, while higher-order conditional entropies decrease or stabilize. This means larger tokenizer training sets produce more diverse marginal token distributions, but the resulting token streams become more predictable once local context is observed.

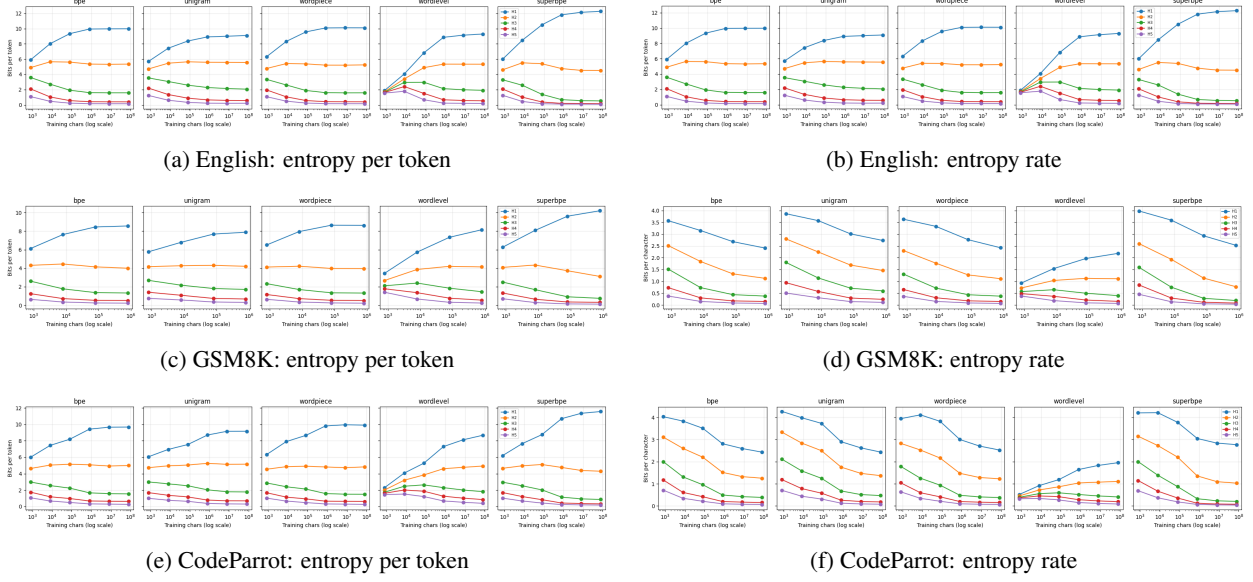


Figure 2: Empirical k -gram entropy for 16k-vocabulary tokenizers across English, math, and code. Left-column plots show entropy per token, while right-column plots show entropy rate in bits per character.

SuperBPE follows the same broad redistribution pattern, but with a different profile. In the entropy-per-token plots, SuperBPE has the highest unigram entropy at large training scales across all three domains. This supports the first part of Hypothesis 2: because SuperBPE can merge across whitespace, it produces more specific phrase-level tokens and spreads probability mass across a larger set of token types. However, its higher-order conditional entropies remain comparable to the other learned tokenizers rather than increasing sharply. Thus, the results do not show that SuperBPE uniformly lowers H_2 through H_5 , but they do show that it maintains competitive higher-order entropy despite much higher H_1 .

The entropy-rate plots make this tradeoff clearer. For BPE, Unigram, WordPiece, and SuperBPE, entropy per character generally decreases as tokenizer training scale increases. So even when unigram entropy rises per token, uncertainty per character can fall because each token represents more text. SuperBPE is especially interesting here: its high entropy per token is paired with a competitive entropy rate, suggesting that its larger cross-whitespace units offset the increased unpredictability of individual token identities.

Overall, these results support Erdogan et al.’s (4) entropy redistribution claim and provide evidence for our SuperBPE hypothesis with an important qualification. SuperBPE clearly exhibits higher unigram entropy than the other learned tokenizers, but its higher-order entropies are better described as comparable rather than uniformly lower. Therefore, the main takeaway is that SuperBPE induces a different entropy-compression tradeoff: it shifts more structure into individual phrase-level tokens while preserving competitive sequential predictability across tokens. The 64k-vocabulary results are included in Appendix A and show the same qualitative behavior.

4.3 Out of Domain

Lastly we evaluate how tokenizers preform when exposed to data outside of the domain of its training corpus. Erdogan et al.’s (4) paper suggests that even as the training corpus size increases, if the tokenizer is being evaluated out of domain it’s compression performance does not always increase. Figure 3 clearly illustrates this phenomena. Figure 3 (a) shows that as the training size increases there is little to no increase in compression ratio after 10^6 training samples for all three graphs. Furthermore for training samples larger than 10^7 , the compression ratio actually decreases for Unigram and WordPiece tokenizers. Figure 3 (b) reveals slightly different results for tokenizers with a larger vocab bound of 64k tokens. For most tokenizers in the code and Turkish examples as the training sample size increases the compression ratio also increases. However after 10^6 training samples there is a decrease in the rate of compression change causing diminishing returns for increasing the training sample size. Additionally, for tokenizers evaluated on Chinese there is no increase in compression ratio as the training sample size is increased.

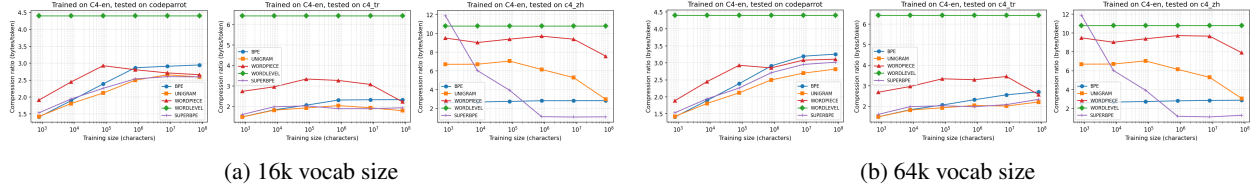


Figure 3: Compression results for tokenizers trained on English evaluated on Code, Turkish, and Chinese

Extending the domain mismatch compression evaluation to SuperBPE yielded mixed results. For both the 16k and 64k vocab sizes, SuperBPE closely tracked the corresponding BPE tokenizer when evaluated on code or Turkish. However, when SuperBPE was evaluated on Chinese, the compression ratio started as the highest of all the tokenizers and as the training size increased the compression ratio dropped to the lowest of all the tokenizers.

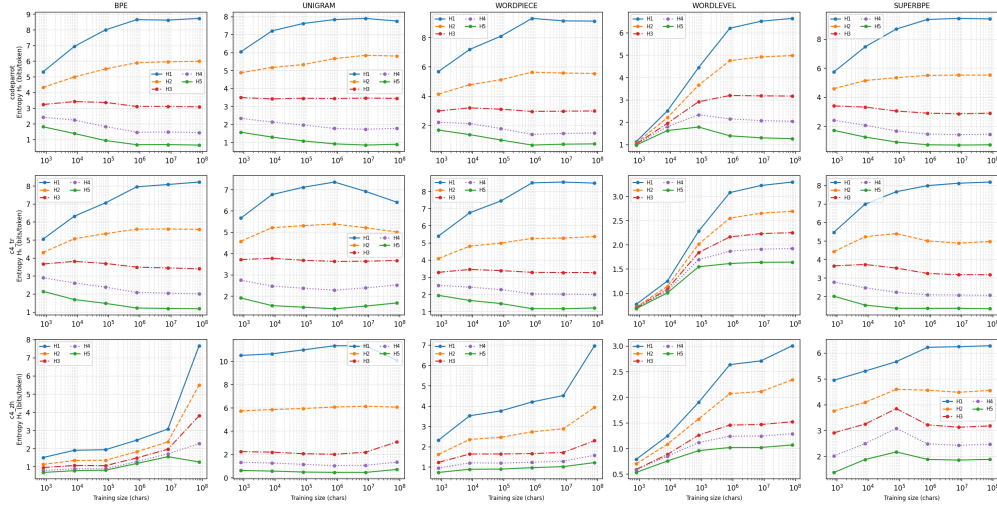


Figure 4: k-gram entropy results for vocab size of 16k trained on English evaluated on code, Turkish, and Chinese

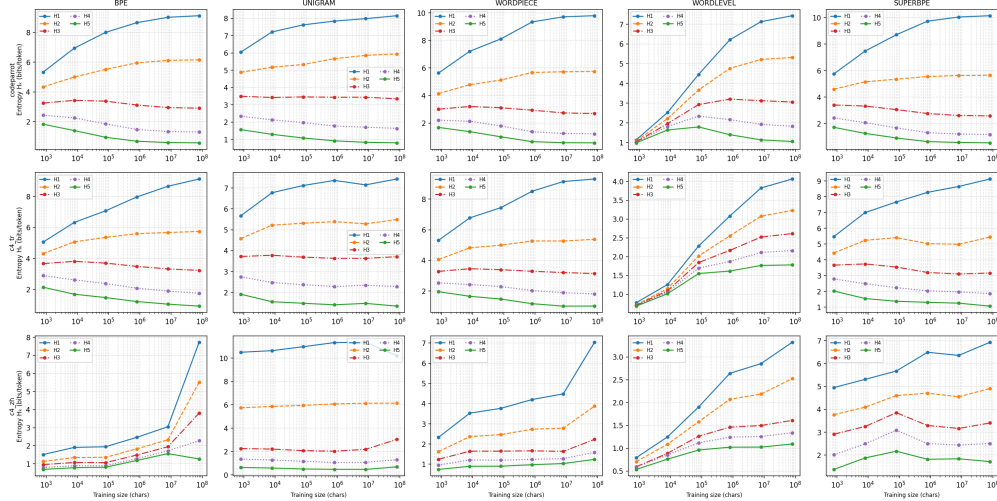


Figure 5: k-gram entropy results for vocab size of 64k trained on English evaluated on code, Turkish, and Chinese

The entropy results show a similar pattern. Figures 4 and 5 show that increasing tokenizer training size does not consistently reduce higher-order entropy under domain mismatch, confirming the result from Erdogan et al. (4) Several

tokenizers show unstable or increasing entropy trends as training scale grows. Furthermore, SuperBPE again does not uniformly degrade more than BPE or WordPiece. This suggests that cross-whitespace tokenization does exhibit a performance drop for out of domain test data but is not necessarily worse than that of tokenizers in all settings.

For completeness, we also evaluate the reverse mismatch direction, where tokenizers are trained on CodeParrot and tested on English, Turkish, and Chinese. These additional compression and entropy results are included in Appendix B. The same broad conclusion holds: tokenizer gains are more reliable in matched-domain settings than under cross-domain evaluation, and SuperBPE generally tracks BPE more closely than our initial hypothesis predicted.

5 Discussion

Our results broadly support the main information-theoretic claims from Erdogan et al. (4) We reproduce the entropy redistribution pattern: as tokenizer training scale increases, unigram entropy tends to rise while higher-order conditional entropies decrease or stabilize. This pattern appears across English, math, and code, suggesting that larger tokenizer training sets produce more diverse token inventories while making token sequences more predictable given local context. We also reproduce the central domain mismatch finding: improvements from tokenizer scale are less reliable when the tokenizer is evaluated outside its training domain.

The SuperBPE extension gives a more nuanced result. We initially expected SuperBPE to be more sensitive to domain mismatch because cross-whitespace tokens may encode domain-specific phrase patterns. Instead, SuperBPE often tracks BPE and WordPiece closely under mismatch, especially when evaluated on code and Turkish. This suggests that cross-whitespace tokenization does not automatically make tokenizers less transferable. However, SuperBPE behaves differently on Chinese, where compression decreases sharply as training size increases. This indicates that SuperBPE’s benefits may depend strongly on whether its learned phrase-level units transfer to the target domain.

The clearest SuperBPE result is its entropy-compression tradeoff. SuperBPE achieves strong compression at larger training scales, especially with a 64k vocabulary, but it also produces much higher unigram entropy than the other learned tokenizers. Its higher-order entropies remain comparable rather than uniformly lower. Therefore, SuperBPE should not be interpreted as a strict improvement over BPE or WordPiece. Instead, it induces a different statistical structure: more information is packed into individual phrase-level tokens, while sequential predictability across tokens remains competitive.

5.1 What was Easy

Overall implementing the methods described in Erdogan et al’s paper provided little challenge. For each evaluation metric the paper clearly documented equations, choices, and the underlying motivation. The paper provides all parameters and clearly details when each configuration was utilized thus eliminating potential doubt about the experimental setup. Additionally, as all of our data and tokenizers were readily available we had ease integrating them into our pipeline. The HuggingFace tokenizers library allowed for easy implementation of the tokenizers pulled from HuggingFace (Unigram, BPE, WordLevel, WordPiece). Additionally, as our data was sourced from HuggingFace we were easily able to load, preprocess, normalize, and save our data.

Furthermore, the availability of the SuperBPE tokenizer codebase, along with good documentation, allowed a significant reduction in implementation difficulties. The documentation clearly outlines the setup as well as procedures to train and extend the vocabulary of a SuperBPE tokenizer. This allowed our group to focus our efforts on recreating the experimental setup rather than spending time recreating already existing tokenizers. SuperBPE also allows for the creation of a HuggingFace tokenizer once training is complete. This led to a seamless integration of SuperBPE with our other HuggingFace tokenizers allowing for a more straight forward evaluation pipeline to be constructed.

As a result, building an end to end pipeline that constructs and evaluates each tokenizer on metrics such as compression ratio, k-gram entropy, and k-gram entropy rate was achievable due to the preexisting, and accessible, datasets, tokenizers, and codebases.

5.2 What was Difficult

While tokenizers generally require low compute we still ran into issues when training or evaluating tokenizers on large datasets. When evaluating and calculating the compression ratio, k-gram entropy, and k-gram entropy rate on test set of 10M characters we ran into limitations where each batch of tokenizers would take over one hour to evaluate. This coupled with the over 160 tokenizers we evaluated (on multiple metrics) led to a difficulty of producing results in an efficient manner.

Another challenge we ran into was the GSM8K dataset only has approximately 3M characters. This meant that we could not train the GSM8K tokenizers on training sizes larger than 10^6 thus yielding incomplete results for thier evaluations. Additionally, due to use using a testing size of 10M characters we could not math as an evaluation or training dataset for the domain mismatch experiments.

5.3 Recommendations for Future Work

Future work should evaluate whether the entropy and compression patterns observed here translate to downstream language model behavior. Our results show that SuperBPE changes the statistical structure of the token stream, but they do not directly show whether this improves language modeling loss, generation quality, or task performance. A natural next step would be to train small language models using BPE and SuperBPE tokenizers and compare whether lower entropy rate or stronger compression predicts downstream gains.

A second direction is to study SuperBPE’s cross-whitespace merges more directly. Instead of only measuring aggregate entropy and compression, future work could inspect which phrase-level tokens are learned, how domain-specific they are, and whether they transfer across domains. This would help explain why SuperBPE behaves similarly to BPE on some mismatch settings but degrades sharply on Chinese.

Finally, future work should expand the mismatch evaluation to more languages and structured domains. Our results suggest that domain mismatch is not a single phenomenon: code, Turkish, Chinese, and math each stress tokenizers in different ways. A larger evaluation across scripts, morphology types, and technical domains would give a clearer picture of when tokenizer scale and cross-whitespace tokenization help or hurt.

References

- [1] Stanley F. Chen and Joshua T. Goodman. An empirical study of smoothing techniques for language modeling, 1996. URL: <https://arxiv.org/abs/cmp-lg/9606011>, arXiv:cmp-lg/9606011.
- [2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL: <https://arxiv.org/abs/2110.14168>, arXiv:2110.14168.
- [3] CodeParrot. codeparrot/codeparrot-clean. <https://huggingface.co/datasets/codeparrot/codeparrot-clean>, 2022. Hugging Face dataset. Accessed: 2026-06-05.
- [4] Mete Erdogan, Abhiram Gorle, Shubham Chandak, Mert Pilanci, and Tsachy Weissman. An information-theoretic perspective on llm tokenizers, 2026. URL: <https://arxiv.org/abs/2601.09039>, arXiv:2601.09039.
- [5] Hugging Face. Hugging face tokenizers. <https://github.com/huggingface/tokenizers>, 2026. Accessed: 2026-06-05.
- [6] Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates, 2018. URL: <https://arxiv.org/abs/1804.10959>, arXiv:1804.10959.
- [7] Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Patrick von Platen, Suraj Patil, Julien Chaumond, Mariama Drame, Julien Plu, Lewis Tunstall, Joe Davison, Mario Šaško, Gunjan Chhablani, Bhavitvya Malik, Simon Brandeis, Teven Le Scao, Victor Sanh, Canwen Xu, Nicolas Patry, Angelina McMillan-Major, Philipp Schmid, Sylvain Gugger, Clément Delangue, Théo Matussière, Lysandre Debut, Stas Bekman, Pierric Cistac, Thibault Goehringer, Victor Mustar, François Lagunas, Alexander M. Rush, and Thomas Wolf. Datasets: A community library for natural language processing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 175–184. Association for Computational Linguistics, 2021. URL: <https://aclanthology.org/2021.emnlp-demo.21/>.
- [8] Alisa Liu, Jonathan Hayase, Valentin Hofmann, Sewoong Oh, Noah A. Smith, and Yejin Choi. Superbpe: Space travel for language models, 2025. URL: <https://arxiv.org/abs/2503.13423>, arXiv:2503.13423.
- [9] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2023. URL: <https://arxiv.org/abs/1910.10683>, arXiv:1910.10683.
- [10] Mike Schuster and Kaisuke Nakajima. Japanese and korean voice search. In *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5149–5152, 2012. doi:10.1109/ICASSP.2012.6289079.
- [11] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units, 2016. URL: <https://arxiv.org/abs/1508.07909>, arXiv:1508.07909.
- [12] The Unicode Consortium. The unicode standard. <https://www.unicode.org/versions/>, 2025. Accessed: 2026-06-05.

Appendix

A Entropy Results

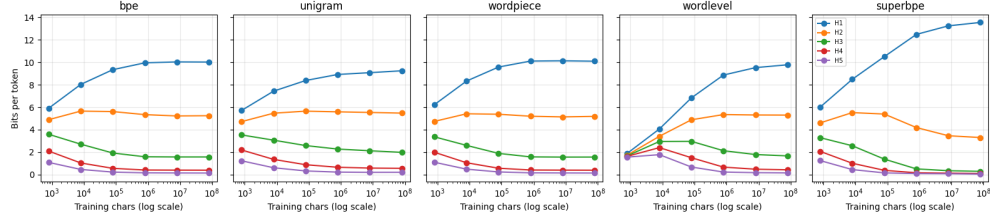


Figure 6: k-gram entropy results for vocab size of 64k trained and evaluated on English

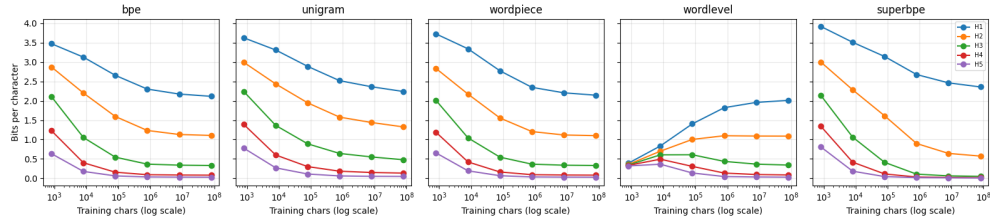


Figure 7: k-gram entropy rate results for vocab size of 64k trained and evaluated on English

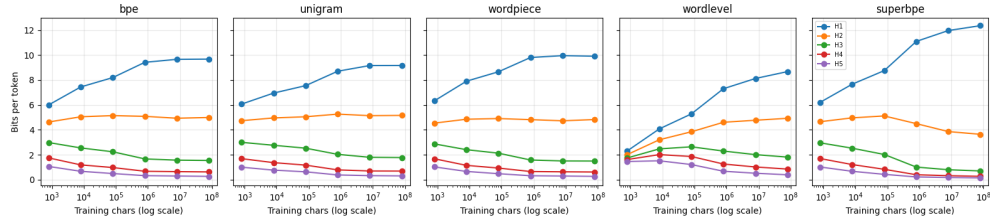


Figure 8: k-gram entropy results for vocab size of 64k trained and evaluated on code

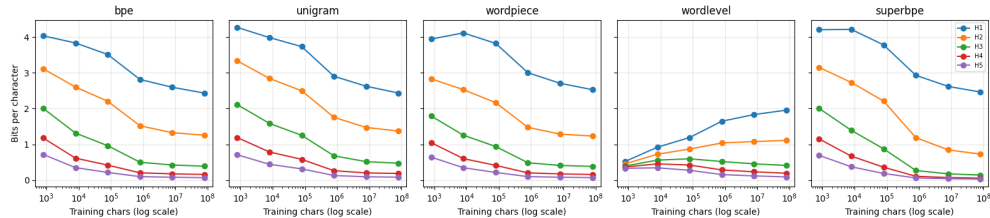


Figure 9: k-gram entropy rate results for vocab size of 64k trained and evaluated on code

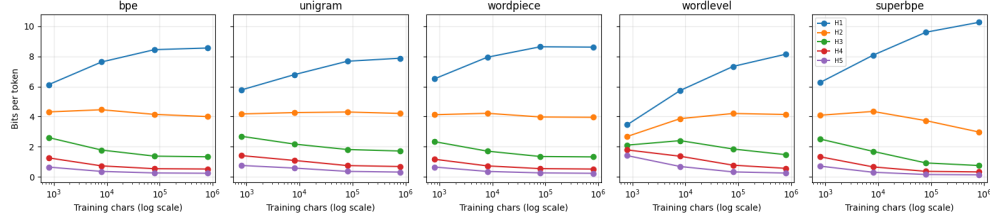


Figure 10: k-gram entropy results for vocab size of 64k trained and evaluated on math

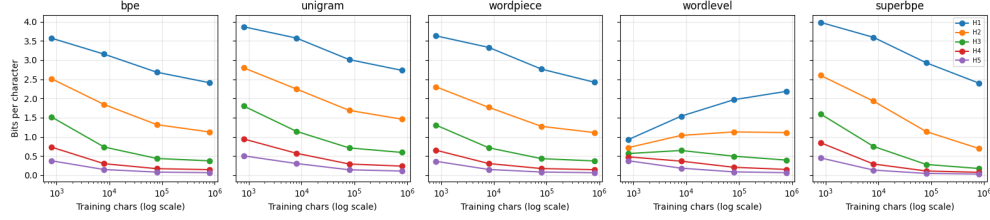


Figure 11: k-gram entropy rate results for vocab size of 64k trained and evaluated on math

B Domain Mismatch

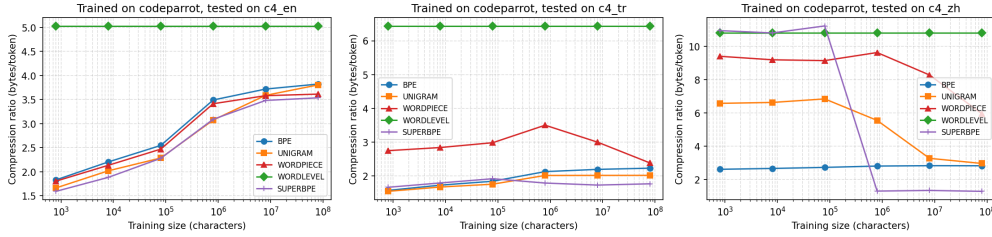


Figure 12: Compression results for vocab size of 16k trained on code evaluated on English, Turkish, and Chinese

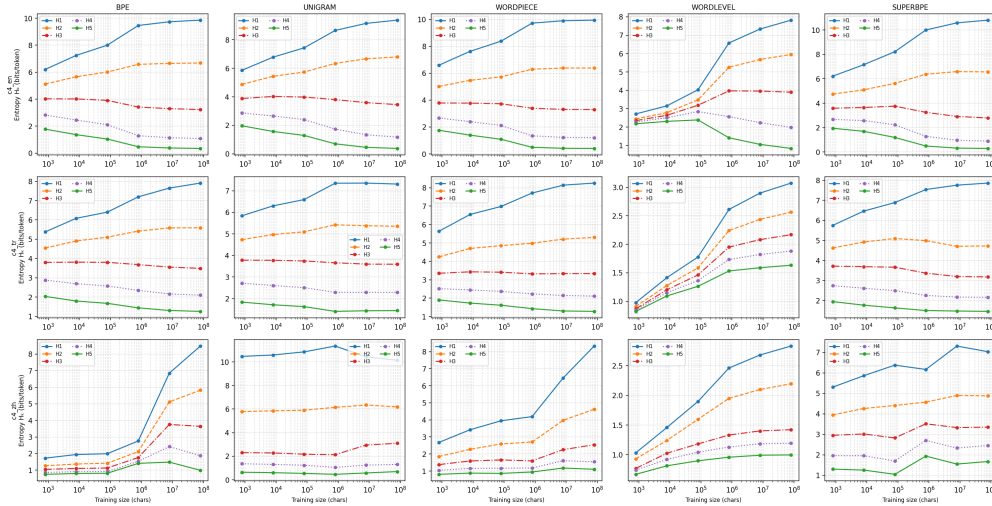


Figure 13: k-gram entropy results for vocab size of 16k trained on code evaluated on English, Turkish, and Chinese

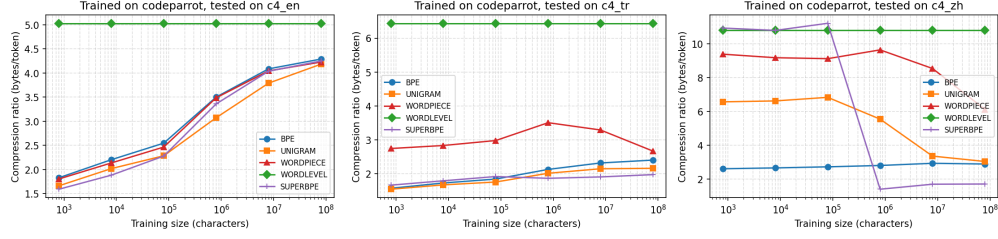


Figure 14: Compression results for vocab size of 64k trained on code evaluated on English, Turkish, and Chinese

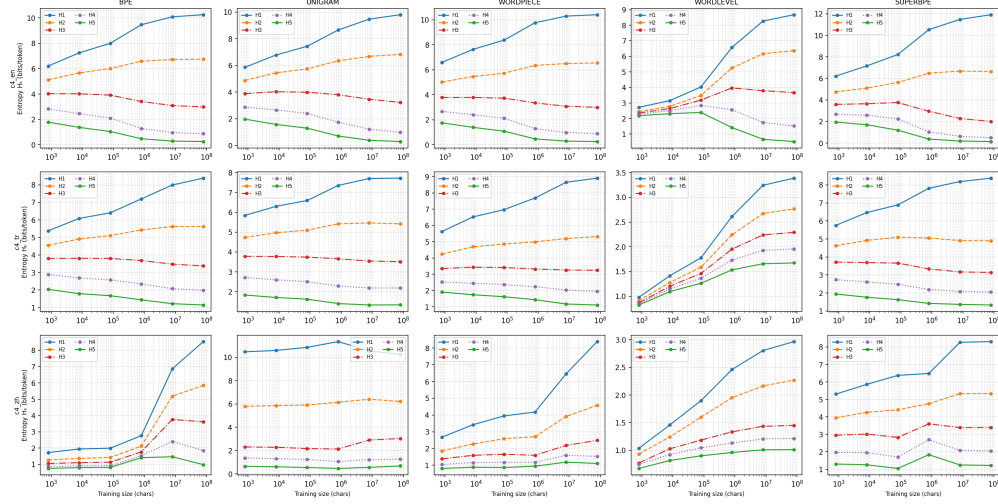


Figure 15: k-gram entropy results for vocab size of 64k trained on code evaluated on English, Turkish, and Chinese